

파이썬 자료구조와 함수

01

자료구조와 반복문





1 파이썬 자료구조

리스트

✔ 리스트 (list) = 목록

» 0에서 시작하는 순서있는 목록

» 자료들을 모아서 사용할 수 있게 해 줌

» **대괄호 []** 내 에 자료들 넣어 선언

✔ 여러 가지 자료를 저장할 수 있는 자료

» 숫자, 문자, 혹은 숫자 또는 문자 혼용



1 파이썬 자료구조

➤ 리스트

✓ 요소 (element)

» 리스트의 대괄호 내부에 넣는 자료

```
print(3)
```

```
print([1,2,3,4,5])
```

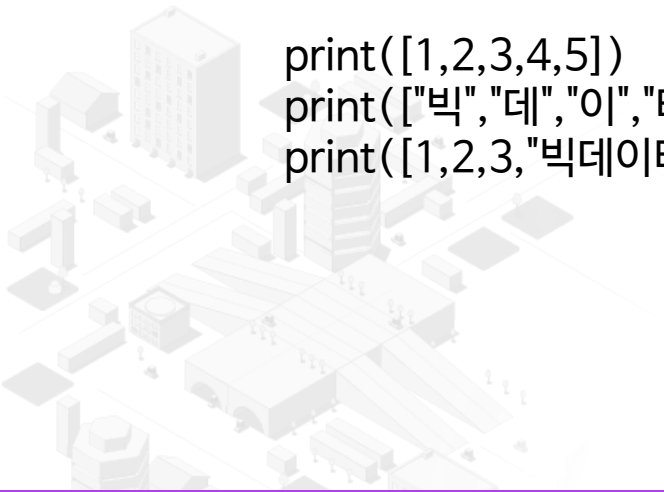
```
print(["빅", "데", "이", "터"])
```

```
print([1,2,3,"빅데이터",True])
```

#숫자만 구성

#문자열 구성

#여러 자료형 구성





1 파이썬 자료구조

리스트

✓ 리스트 내부의 요소 각각 사용

» 리스트 이름 뒤 대괄호 [] 입력 후 인덱스 지정

» 인덱스: 대괄호 안에 들어간 숫자, 자료의 위치 나타내는 숫자

1	2	3	빅데이터	True
[0]	[1]	[2]	[3]	[4]



1 파이썬 자료구조

➤ 리스트

✔ 리스트 내부의 요소 각각 사용

```
a = [1,2,3,"빅데이터",True]
```

```
print("a[0] :", a[0])
```

```
print("a[1] :", a[1])
```

```
print("a[2] :", a[2])
```

```
print("a[3] :", a[3])
```

```
print("a[4] :", a[4])
```



```
a[0] : 1
```

```
a[1] : 2
```

```
a[2] : 3
```

```
a[3] : 빅데이터
```

```
a[4] : True
```



1 파이썬 자료구조

리스트

```
a = ['GIS', 'JAVA', 'DBMS', 1, 2, True]
print(a[0])
print(a[1])
```

```
#대괄호 안에 음수 넣어 뒤에서부터 요소 선택
print(a[-1])
```

```
print(a[3:])
print()
```

```
#리스트 특정 요소를 변경할 수 있음
a[0] = "WEBGIS"
print(a[0])
```

```
#리스트의 길이 넘는 인덱스로 요소에 접근하려는 경우 발생
print(a[6])
#슬라이스를 이용한 인덱스 요소 접근은 오류 발생하지 않음
print(a[5:1000])
```

```
GIS
JAVA
True
[1, 2, True]
```

```
WEBGIS
```

IndexError: list index out of range



1 파이썬 자료구조

리스트

리스트 연산자

문자열 연산자와 비슷

⋯ + : 연결, * : 반복, len() : 요소개수

⋯ in 연산자: 특정 값이 리스트 내부에 있는지 확인

⋯ not in 연산자: 리스트 내부에 해당 값이 없는지 확인





1 파이썬 자료구조

리스트

리스트 연산자

문자열 연산자와 비슷

a = [1,2,3]

b = [4,5,6]

print(a + b)

print(a * 2)

print(2 * a)

print(len(a))

print(10 in a)

print(10 not in a)

[1, 2, 3, 4, 5, 6]

[1, 2, 3, 1, 2, 3]

[1, 2, 3, 1, 2, 3]

3

False

True



1 파이썬 자료구조

리스트

리스트 연산자

» 리스트 연결 연산자 사용

💬 비파과적 처리 : 연산 실행 전후 원본에 변화 없음

```
a = [1,2,3]
```

```
b = [4,5,6]
```

```
print(a + b)
```



```
[1, 2, 3, 4, 5, 6]
```

```
print(a)
```

```
print(b)
```

```
[1, 2, 3]
```

```
[4, 5, 6]
```



1 파이썬 자료구조

리스트

✔ 리스트 요소추가

» append() 함수: 리스트 뒤에 요소 추가

» insert(추가위치, 추가 값) 함수: 리스트 특정위치에 요소 추가

» extend() 함수

💬 원래 리스트 뒤에 새로운 **리스트**의 모든 요소 추가



1 파이썬 자료구조

리스트

✔ 리스트 요소추가

```
a = [1,2,3]
```

```
a.append(4)
```

```
print(a)
```

[1, 2, 3, 4]

```
a.insert(1,1)
```

```
print(a)
```

[1, 1, 2, 3, 4]

```
a.extend([5,6,7])
```

```
print(a)
```

[1, 1, 2, 3, 4, 5, 6, 7]



1 파이썬 자료구조

리스트

✔ 리스트 요소추가

» 파과적 처리

💡 함수 실행 전후 원본 리스트 자체에 변화 생김

```
a = [1,2,3]
```

```
b = [4,5,6]
```

```
a.extend(b)
```

```
print(a)
```

```
print(b)
```



```
[1, 2, 3, 4, 5, 6]
```

```
[4, 5, 6]
```

```
a.append(7)
```

```
print(a)
```

```
[1, 2, 3, 4, 5, 6, 7]
```



1 파이썬 자료구조

리스트

✔ 리스트 요소제거

» 인덱스로 제거하기

💬 del 키워드

```
a = [1,2,3,4,5,6,7]
```

```
del a[1]
```

```
print(a)
```

```
# a[0], a[1], a[2] 삭제
```

```
del a[0:3]
```

```
print(a)
```

[1, 3, 4, 5, 6, 7]

[5, 6, 7]



1 파이썬 자료구조

리스트

✓ 리스트 요소제거

» 인덱스로 제거하기

💬 pop() 함수

```
a = [1,2,3,4,5,6,7]
```

```
a.pop(1)
```

```
print(a)
```

```
a.pop() #a.pop(-1) 마지막 요소제거
```

```
print(a)
```

[1, 3, 4, 5, 6, 7]

[1, 3, 4, 5, 6]



1 파이썬 자료구조

➤ 리스트

✔ 리스트 요소제거

➤ remove() 함수

💬 값으로 제거하기

```
a = [1,2,3,4,5,6,7]
a.remove(4)
print(a)
```

[1, 2, 3, 5, 6, 7]

```
# 동일한 값은 하나만 제거
b = [1,2,3,4,3,5]
b.remove(3)
print(b)
```

[1, 2, 4, 3, 5]

```
# 리스트 내부의 요소를 모두 제거
b.clear()
print(b)
```

[]



1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 키를 기반으로 값을 저장하는 것

자료형	위치	선언형식	개념
리스트	인덱스	<code>a = []</code>	인덱스 기반으로 값을 저장
딕셔너리	키	<code>a = {}</code>	키 기반으로 값을 저장

리스트 = []
리스트 = [1, 2, 3]
리스트[0] #인덱스

딕셔너리 = {
 "키1":"값"
 "키2":"값"
 "키3":"값"
}
딕셔너리["키1"] #키



1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 딕셔너리 선언

» 중괄호로 선언하며 ‘키: 값’ 형태를 쉼표로 연결

» 딕셔너리 요소 접근

💬 특정 키 값 출력: 딕셔너리 뒤에 대괄호 입력하고 그 내부에 키 입력





1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 딕셔너리 선언

딕셔너리 : {키 : 값}

```
a = {"키1": "값1", "키2": "값2"}
```

print(a)

```
print(a["키1"])
```

```
print(a["키2"])
```

```
print(type(a))
```

#자료형 확인

값1

값2

<class 'dict'>



1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 딕셔너리 요소접근

- » 딕셔너리 내부 값에 문자열, 숫자, 불, 리스트 등 다양한 자료 입력

```
a = {  
    "name": "한국대학",  
    "type": ["공업계열", "사회실무계열", "예능계열"],  
    "dept": ["항공과", "컴정과", "항공운항과", "호경과", "산디과", "패디과"],  
    "addr": "인천 미추홀구 인하로100"  
}
```

```
print(a["name"])  
print(a["type"])  
print(a["dept"])  
print(a["addr"])
```

한국대학
['공업계열', '사회실무계열', '예능계열']
['항공과', '컴정과', '항공운항과', '호경과', '산디과', '패디과']
인천 미추홀구 인하로100



1 파이썬 자료구조

🔍 딕셔너리 (dictionary)

✔ 딕셔너리 요소접근

» 요소 - 리스트 안의 특정 값 출력하려는 경우

```
a = {  
    "name": "한국대학",  
    "type": ["공업계열", "사회실무계열", "예능계열"],  
    "dept": ["항공과", "컴정과", "항공운항과", "호경과", "산디과", "패디과"],  
    "addr": "인천 미추홀구 인하로100"  
}
```

```
print(a["name"])  
print(a["type"][0])  
print(a["dept"][0])  
print(a["addr"])
```

----->

한국대학
공업계열
항공과
인천 미추홀구 인하로100



1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 딕셔너리 요소접근

» 딕셔너리 요소 특정값 변경

```
a["name"] = "K-MOOC"  
print(a["name"])
```





1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 딕셔너리 요소접근

» 딕셔너리와 반복문

```
a = {  
    "name": "한국대학",  
    "type": ["공업계열", "사회실무계열", "예능계열"],  
    "dept": ["항공과", "컴정과", "항공운항과", "호경과", "산디과", "패디과"],  
    "addr": "인천 미추홀구 인하로100"  
}
```

```
for k in a:  
    print("key: ", k)
```

```
for k in a:  
    print(k, ":", a[k])
```

```
# print("{} : {}".format(k, a[k]))
```

key: name
key: type
key: dept
key: addr

name : 한국대학
type : ['공업계열', '사회실무계열', '예능계열']
dept : ['항공과', '컴정과', '항공운항과', '호경과', '산디과', '패디과']
addr : 인천 미추홀구 인하로100



1 파이썬 자료구조

🔍 딕셔너리 (dictionary)

✔ 딕셔너리에 값 추가

➤ 딕셔너리에 값 추가할 때는 키를 기반으로 값 입력

```
a = {  
    "name": "한국대학",  
    "type": ["공업계열", "사회실무계열", "예능계열"],  
    "dept": ["항공과", "컴정과", "항공운항과", "호경과", "산디과", "패디과"],  
    "addr": "인천 미추홀구 인하로100"  
}
```

```
a["tel"] = "032-870-1234"
```

```
for i in a:
```

```
    #print(i,":", a[i])
```

```
    print("{} : {}".format(i, a[i]))
```

name : 한국대학

type : ['공업계열', '사회실무계열', '예능계열']

dept : ['항공과', '컴정과', '항공운항과', '호경과', '산디과', '패디과']

addr : 인천 미추홀구 인하로100

tel : 032-870-1234



1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 딕셔너리에 값 제거

» 딕셔너리 요소의 제거 : del 키워드

```
a = {  
    "name": "한국대학",  
    "type": ["공업", "사회", "예능"],  
    "dept": ["항기과", "컴정과", "호경과"],  
    "addr": "인천시 미추홀구 인하로 100"  
}
```

```
a["tel"] = "032-870-2114"  
a["addr"]
```

```
for i in a:  
    if type(a[i]) is list:  
        for j in a[i]:  
            print(i, ":", j)  
    else:  
        print(i, ":", a[i])
```

리스트 요소 출력

addr 요소 제거

```
name : 한국대학  
type : 공업  
type : 사회  
type : 예능  
dept : 항기과  
dept : 컴정과  
dept : 호경과  
tel : 032-870-2114
```



1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 딕셔너리 키 확인

» KeyError 예외

💬 딕셔너리에서 존재하지 않는 키에 접근할 경우

```
a = {  
    "name": "한국대학",  
    "type": ["공업계열", "사회실무계열", "예능계열"],  
    "dept": ["항공과", "컴정과", "항공운항과", "호경과", "산디과", "패디과"],  
    # "addr": "인천 미추홀구 인하로100"  
}
```

`print(a["addr"])` ----- ➔ *KeyError: addr*



1 파이썬 자료구조

🔧 딕셔너리 (dictionary)

✔ 딕셔너리 키 확인

» in 키워드

💬 사용자로부터 접근하고자 하는 키 입력 받은 후 존재하는 경우에만 접근하여 값 출력

```
a = {  
    "name": "한국대학",  
    "type": ["공업", "사회", "예능"],  
    "dept": ["항기과", "컴정과", "호경과"]  
    # "addr": "인천시 미추홀구 인하로 100",  
}  
print(a["name"])  
  
if "addr" in a:  
    print(a["addr"])  
else:  
    print("키에러: 키가없음")
```

[출력결과]

한국대학

키가 없음



1 파이썬 자료구조

➤ 딕셔너리 (dictionary)

✔ 딕셔너리 키 확인

» get() 함수

💬 딕셔너리의 키로 값을 추출, 존재하지 않는 키에 접근할 경우 None 출력

```
a = {
    "name": "한국대학",
    "type": ["공업계열", "사회실무계열", "예능계열"],
    "dept": ["항공과", "컴정과", "항공운항과", "호경과", "산디과", "패디과"],
    "addr": "인천 미추홀구 인하로100"
}
print(a["name"])
print("get() 함수:", a.get("name"))
print("get() 함수:", a.get("tel"))
```

```
if a.get("tel") != None:
    print(a.get("tel"))
else:
    print("키가 없음")
```



한국대학
get() 함수: 한국대학
get() 함수: None
키가 없음



1 파이썬 자료구조

🔗 딕셔너리 (dictionary)

✔ 딕셔너리 키 확인

» NameError 오류

💬 name이라는 이름이 정의되지 않음

```
a = {
```

큰 따옴표 없음

```
    name:"한국대학",  
    "addr": "인천 미추홀구 인하로100"  
}
```

```
print(a["name"])  
print(a["addr"])
```

NameError: name 'name' is not defined



1 파이썬 자료구조

🔧 딕셔너리 (dictionary)

✔ 딕셔너리 키 확인

» NameError 오류

💬 name 이름을 변수로 만들어 해결

```
name = "이름"
a = {
    name:"한국대학",
    "addr":"인천 미추홀구 인하로100"
}
print(a)
print(a[name])
```

{'이름': '한국대학', 'addr': '인천 미추홀구 인하로100'}

한국대학



1 파이썬 자료구조

범위 (range)

- ✔ 특정 횟수만큼 반복해서 돌리고 싶을 때
for 반복문과 조합하여 사용

매개변수	의미	예시
숫자를 한 개 넣는 방법	0부터 A-1까지의 정수로 범위 만들	range(A)
숫자를 두 개 넣는 방법	A부터 B-1까지의 정수로 범위 만들	range(A, B)
숫자를 세 개 넣는 방법	A부터 B-1까지의 정수로 범위 만들되 앞뒤의 숫자가 C만큼의 차이 가짐	range(A, B, C)



1 파이썬 자료구조

범위 (range)

```
a = range(0,10,1)
```

```
print(a)
```

```
# 0에서 9까지 1씩 증가
```

```
a = range(0,10,1) #0 -9까지 범위
```

```
print(list(a))
```

range(0, 10)

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```
a = range(0,10)
```

```
print(list(a))
```

```
# 마지막 매개변수 생략
```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```
a = range(10)
```

```
print(list(a))
```

```
# 처음 매개변수 생략: 0부터
```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```
b = range(0, 10, 2)
```

```
print(list(b))
```

[0, 2, 4, 6, 8]

```
c = range(0, 10, 3)
```

```
print(list(c))
```

[0, 3, 6, 9]



1 파이썬 자료구조

범위 (range)

✓ 매개변수로 나눗셈을 사용한 경우 오류 발생

```
a = range(0, 10 / 2)
print(list(a))
```

매개변수 : 정수

TypeError: 'float' object cannot be interpreted as an integer

정수 나누기 연산자

```
a = range(0, 10 // 2)
print(list(a))
```

[0, 1, 2, 3, 4]



1 파이썬 자료구조

범위 (range)

✓ for 반복문 조합

```
for i in range(0, 10, 1):  
    print(i)
```

특정회수 만큼 반복

```
for i in range(5):  
    print(i)
```

0
1
2
3
4

0
1
2
3
4
5
6
7
8
9



1 파이썬 자료구조

범위 (range)

- ✓ 키와 값으로 이루어진 각 리스트를 조합해 하나의 딕셔너리를 만들기

```
k = ["name", "system", "tel", "addr"]
```

```
v = ["한국대학", 2, "032-870-0000", "인천 미추홀구"]
```

```
a = {}
```

```
for i in range(len(k)):
```

```
    a[k[i]] = v[i]
```

```
print(a)
```

```
{'name': '한국대학', 'system': 2, 'tel': '032-870-0000', 'addr': '인천 미추홀구'}
```



1 파이썬 자료구조

튜플 (tuple)

- ✓ 함수와 함께 많이 사용되는 리스트와 비슷한 자료형, but 소괄호 () 이용 선언
- ✓ 한번 결정된 요소를 바꿀 수 없다는 점이 리스트와 다름





1 파이썬 자료구조

➤ 튜플 (tuple)

#리스트

```
a = [1,2,3,4]
```

```
print(a[0])
```

```
print(a[1])
```

```
print(a[2])
```

```
print(a[3])
```

```
a[0] = 100
```

```
print(a[0])
```

```
a = (1,2,3,4)
```

a = 1,2,3,4 #튜플

```
print(a[0])
```

```
print(a[1])
```

```
print(a[2])
```

```
print(a[3])
```

1

2

3

4

```
a = (1,2,3,4)
```

```
a[0] = 100
```

TypeError: 'tuple' object does not support item assignment



1 파이썬 자료구조

튜플 (tuple)

✓ 튜플 사용 : 한번에 여러 개 할당

```
[a, b] = [10, 20]
```

#리스트

```
(c, d) = (10, 20)
```

#튜플

```
print(a,b,c,d)
```

괄호없이 사용가능

```
c, d = 10, 20
```

```
print(c,d)
```



1 파이썬 자료구조

▶ 튜플 (tuple)

✓ 리스트 반복문 출력

```
a = [10,2,33,23,15]
```

튜플 (괄호 생략가능)

```
for (i, v) in enumerate(a):  
    print(i, "번째 요소 값: ", v)
```

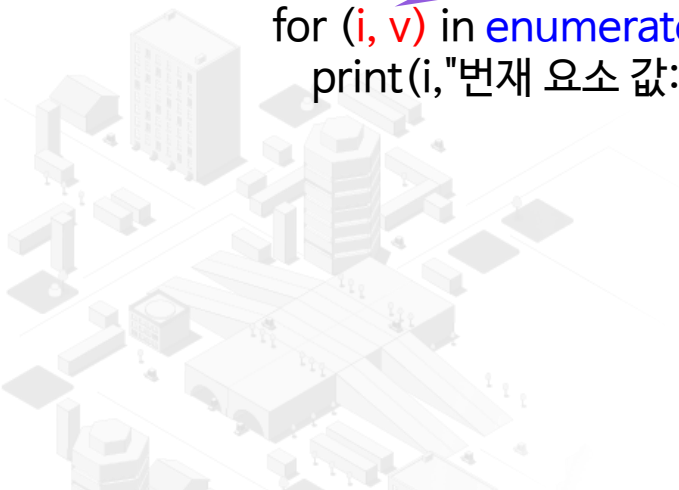
0 번째 요소 값: 10

1 번째 요소 값: 2

2 번째 요소 값: 33

3 번째 요소 값: 23

4 번째 요소 값: 15





2 반복문

for 반복문

✔ 특정 코드를 반복해서 실행할 때 사용하는 기본 구문

✔ 문자열, 리스트, 딕셔너리 등과 조합

a = [1,2,3,4,5,6,7]

```
print(a)
```

[1, 2, 3, 4, 5, 6, 7]

```
for i in a:  
    print(i)
```

1
2
3
4
5
6
7

```
for i in a:  
print(i)
```

IndentationError: expected an indented block



2 반복문

for 반복문

✓ 반복문 활용

» 홀수 짝수 구분하기

```
a = [100, 9, 21, 4, 5, 43, 22, 214]
```

```
for i in a:  
    if i%2==0:  
        print("{} : 짝수".format(i))  
    else:  
        print("{} : 홀수".format(i))
```

```
100 : 짝수  
9 : 홀수  
21 : 홀수  
4 : 짝수  
5 : 홀수  
43 : 홀수  
22 : 짝수  
214 : 짝수
```



2 반복문

for 반복문

✓ 반복문 활용

» 홀수 짝수 구분하기

```
a = [100, 9, 21, 4, 5, 43, 22, 214]
```

```
b = ["짝수", "홀수"]
```

```
print(b[0]) # 짝수
```

```
print(b[1]) # 홀수
```

```
print(b[100 % 2])
```

```
print(b[9 % 2])
```

```
for i in a:
```

```
    print(i, ":", b[i % 2])
```

100 : 짝수

9 : 홀수

21 : 홀수

4 : 짝수

5 : 홀수

43 : 홀수

22 : 짝수

214 : 짝수

b[0] = "짝수" b[1] = "홀수"



2 반복문

⚙ while 반복문

- ✓ 리스트 또는 딕셔너리 내부의 요소를 특정 횟수만큼 반복

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

0
1
2
3
4

- ✓ 참고: 강제 반복 종료

» KeyboardInterrupt : **ctrl + c**



2 반복문

▶ while 반복문

✔ 리스트 내부에서 해당하는 값을 여러 개 제거

» while 반복문의 조건을 '리스트 내부에 요소가 있는 동안'으로 지정

```
a = [1, 2, 3, 1, 2, 3, 1, 2, 3]
a.remove(1) # 맨처음 1삭제
print(a)
```

[2, 3, 1, 2, 3, 1, 2, 3]

```
# 모든 1 삭제
while 1 in a:
    a.remove(1)
print(a)
```

[2, 3, 2, 3, 2, 3]

```
a = [1, 2, 3, 1, 2, 3, 1, 2, 3]

while 1 in a:
    a.remove(1)
    print(a)
```

[2, 3, 1, 2, 3, 1, 2, 3]
[2, 3, 2, 3, 1, 2, 3]
[2, 3, 2, 3, 2, 3]



2 반복문

▶ while 반복문

✓ 시간 관련된 기능 가져오기: timestamp

» 1970년 1월 1일 0시 0분 0초부터의 경과 시간

특정시간 대기하는 프로그램

```
import time
```

```
a = time.time()  
print(a)
```

```
while a + 3 > time.time():  
    pass
```

```
print("시간 대기 종료")
```

실행할 때마다 결과다름

1618209209.0130596

(잠시 대기 후)

시간 대기 종료



2 반복문

▶ while 반복문

✔ break 키워드

» 반복문 벗어날 때 사용하는 키워드

```
i = 0
```

```
while True:
```

```
    print(i, "번째 반복")
```

```
    i += 1
```

```
    a = input(">> 종료(y): ")
```

```
    if a in ["Y", "y"]:
```

```
        print("반복 종료")
```

```
        break
```

0 번째 반복

>> 종료(y)

1 번째 반복

>> 종료(y)

2 번째 반복

>> 종료(y)y

반복 종료



2 반복문

➤ while 반복문

✔ continue 키워드

➤ 현재 반복을 생략하고 다음 반복으로 넘어감, 들여쓰기 단계를 줄여줄 수 있음

```
a = [1, 10, 3, 20, 4, 45]
```

```
for i in a:
    if i < 10:
        continue
    print(i)
```

#들여쓰기 하지 않음

10
20
45

```
for i in a:
    if i >= 10:
        print(i)
```

```
for i in a:
    if i < 10:
        pass
    else:
        print(i)
```

파이썬 자료구조와 함수

02

함수와 파일처리



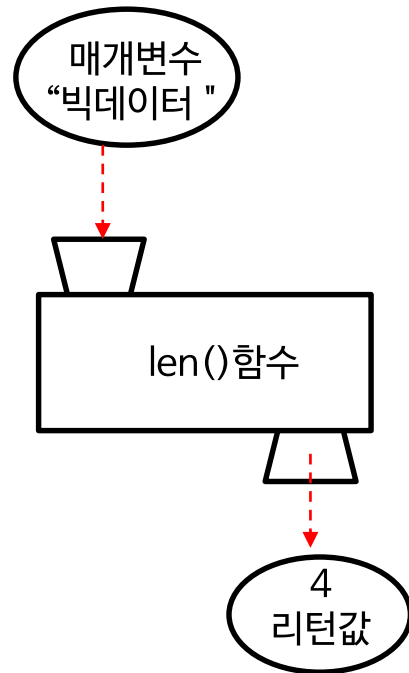


1 함수

▶ 함수 기본 개념

✔ 함수 = 코드의 집합

구분	주요 내용
함수호출	함수 사용
매개변수	함수 호출 시 괄호 내부에 넣는 여러 가지 자료
리턴값	함수를 호출하여 최종적으로 나오는 결과





1 함수

▶ 함수 기본 개념

함수 선언

```
def f1():  
    print("함수호출")  
    print("함수호출")  
    print("함수호출")
```

함수호출
함수호출
함수호출

함수 호출

f1()





1 함수

➤ 함수 기본 개념

✔ (일반)매개변수

함수 선언

```
def f1(a,b,c):  
    print(str(a) + "함수호출")  
    print(str(b) + "함수호출")  
    print(str(c) + "함수호출")
```

함수 호출

```
f1(1, 2, 3)
```

1 함수호출

2 함수호출

3 함수호출



1 함수

▶ 함수 매개변수 생성

✔ 매개변수와 관련된 TypeError

#함수선언

```
def f1(a,b,c):  
    print(str(a) + "함수호출")  
    print(str(b) + "함수호출")  
    print(str(c) + "함수호출")
```

#함수호출

```
f1(1)
```

***TypeError:** f1 () missing 2 required positional arguments: 'b' and 'c'*



1 함수

▶ 함수 매개변수 생성

✔ 매개변수와 관련된 TypeError

```
def f1(str, n):  
    for i in range(n):  
        print(str)
```

#문자열 3회 반복
f1("함수호출", 3)

#매개변수 1개 입력
f1("함수호출")

#매개변수 3개 입력
f1("함수호출", 3, 100)

함수호출
함수호출
함수호출

*TypeError: f1 () missing
1 required positional argument: 'n'*

*TypeError: f1 () takes 2 positional arguments
but 3 were given*



1 함수

▶ 함수 매개변수 생성

✔ 기본 매개변수 (default)

- ▶ 매개변수 값 입력하지 않았을 경우 매개변수에 들어가는 기본값
- ▶ 기본 매개변수는 **마지막**에 입력함





1 함수

▶ 함수 매개변수 생성

✔ 기본 매개변수 (default)

#함수선언

```
def f1(a,b,c=5):  
    print(str(a) + "함수호출")  
    print(str(b) + "함수호출")  
    print(str(c) + "함수호출")
```

#함수호출

f1(1,2,3)

f1(1,2)

1함수호출

2함수호출

3함수호출

1함수호출

2함수호출

5함수호출



1 함수

▶ 함수 매개변수 생성

✔ 기본 매개변수 (default)

```
def f1(str, n):  
    for i in range(n):  
        print(str)
```

```
f1("빅데이터", 3)  
f1("빅데이터")
```

TypeError

```
def f1(str, n=3):  
    for i in range(n):  
        print(str)
```

```
f1("빅데이터", 3)  
f1("빅데이터")
```

#3회 반복

빅데이터
빅데이터
빅데이터
빅데이터
빅데이터
빅데이터



1 함수

▶ 함수 매개변수 생성

✓ 기본 매개변수가 가변 매개변수보다 앞에 올 때, 기본 매개변수의 의미가 사라짐

» 오류 `def f1(n=2, str)`

» “빅데이터”가 첫번째 매개변수에 들어감

» 실제 반복 회수를 넣어주어야 하므로 기본 매개변수의 의미가 사라짐

```
def f1(n=2, str):  
    for i in range(n):  
        print(str)
```

```
f1("빅데이터")
```

SyntaxError: non-default argument follows default argument



1 함수

▶ 함수 매개변수 생성

✔ 가변 매개변수

» 매개변수를 원하는 만큼 받을 수 있는 함수





1 함수

▶ 함수 매개변수 생성

✔ 제약

▶ 가변 매개변수 뒤에는 일반 매개변수 올 수 없음

▶ 가변 매개변수는 **하나만 사용**할 수 있음

```
def f1(a,b,*c):  
    print(a)           #매개변수  
    print(b)           #매개변수  
    print(c) #가변매개변수
```

f1(0,1,2,3,4,5)

오류: def f1(*c, a,b):

오류: def f1(a,b,*c,*d):

0
1
(2, 3, 4, 5)



1 함수

▶ 함수 매개변수 생성

✔ 일반 매개변수 - 가변 매개변수 - 기본 매개변수 순서로 선언

```
def f1(a, b, *c, d=10, e=20):  
    print(a, b)  
    print(c)  
    print(d, e)
```

기본 매개변수로 값이
전달되지 않음

```
f1(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

0 1

(2, 3, 4, 5, 6, 7, 8, 9, 10)

10 20



1 함수

▶ 함수 매개변수 생성

✔ 키워드 매개변수

» 매개변수 이름을 지정해서 입력하는 매개변수

```
def f1(a, b, *c, d=10, e=20):  
    print(a, b)  
    print(c)  
    print(d, e)
```

```
f1(0,1,2,3,4,5,6,7,8,9,10, d=5)
```

```
0 1  
(2, 3, 4, 5, 6, 7, 8, 9, 10)  
5 20
```



1 함수

리턴

✓ 자료 없이 리턴하기

» return 키워드 : 함수를 실행했던 위치로 돌아가게 함

함수 선언

```
def f1(a, b, c):  
    print("함수호출" + str(a))  
    print("함수호출" + str(b))  
    return  
    print("함수호출" + str(c))
```

#함수 호출

f1(1, 2, 3)

함수호출1

함수호출2



1 함수

리턴

✓ 리턴값 (return value)

» 함수의 결과

함수 선언

```
def f1(a, b, c):  
    print(str(a) + "함수호출")  
    print(str(b) + "함수호출")  
    return a+b+c  
    print(str(c) + "함수호출")
```

#함수 호출 후 출력

```
print(f1(1, 2, 3))
```

1함수호출
2함수호출
6



1 함수

▶ 콜백 함수(callback)

✔ 함수의 매개변수로 함수 전달하기

```
def test():  
    print("테스트")
```

```
test()
```

테스트

#콜백함수
#매개변수 : 값, 함수

```
def f1(a):  
    a()
```

함수 매개변수로 함수
이름 전달

```
def test():  
    print("테스트")
```

```
test()  
f1(test)      테스트
```



1 함수

▶ 람다(lambda) 함수

✔ "lambda 입력 매개변수 : 결과 리턴값"

» lambda는 def와 같은 함수와 동일한 기능

» 한 줄 내에 간편하게 사용 가능

» 기존 def 함수이름(변수): return 결과값 처럼 길고 복잡하게 할 필요가 없음

» 복잡한 함수라면 lambda보다는 기존 함수 사용 권장

» 단순 사칙연산, 간단한 문자열 추출 등 lambda를 적용한 map, apply 함수 사용이 효율적임



1 함수

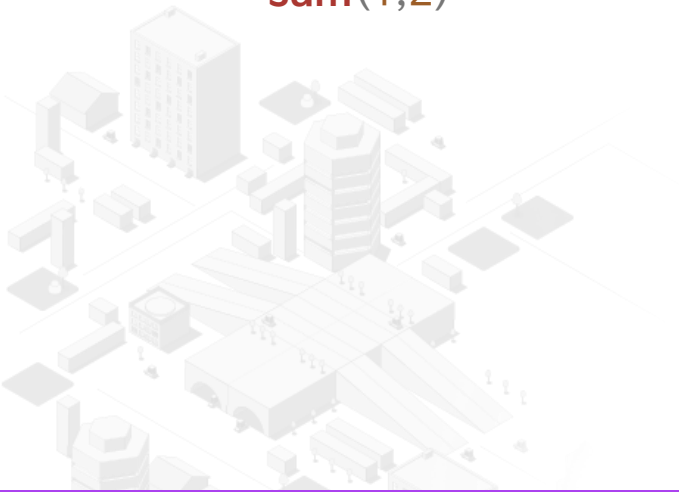
▶ 람다(lambda) 함수

✔ "lambda 입력 매개변수 : 결과 리턴값"

```
def sum(a,b):  
    return a + b  
sum(1,2)
```



```
(lambda a,b : a + b)(1,2)
```





1 함수

▶ 람다(lambda) 함수

✔ 람다 (lambda)

▶ 기능을 매개변수로 전달하는 코드를 더 효율적으로 작성

```
def f1(a):  
    for i in range(5):  
        a(i)
```

```
def test(n):  
    print("테스트", n)
```

```
f1(test)  
f1(lambda n: print("람다테스트", n))
```

람다테스트 0
람다테스트 1
람다테스트 2
람다테스트 3



2 파일처리

▶ 텍스트 파일의 처리

✔ 파일 열기 (open) - 파일 읽기 (read) - 파일 쓰기 (write)

▶ 파일 열고 닫기

✔ open() 함수

✔ closed() 함수

모드	설 명	
w	Write 모드	새로 쓰기 모드
a	Append 모드	뒤에서 이어서 쓰기 모드
r	Read 모드	읽기 모드



2 파일처리

▶ 텍스트 파일의 처리

```
f = open("test.txt", "w")  
  
f.write("내가만든 파일쓰기")  
f.close()
```

```
f = open("test.txt", "r")  
print(f.read())  
f.close()
```

test - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

파일 쓰기



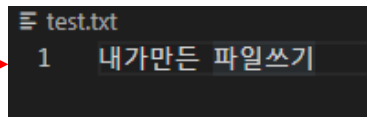
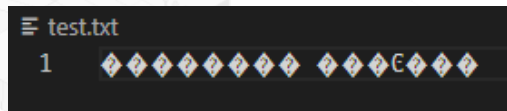
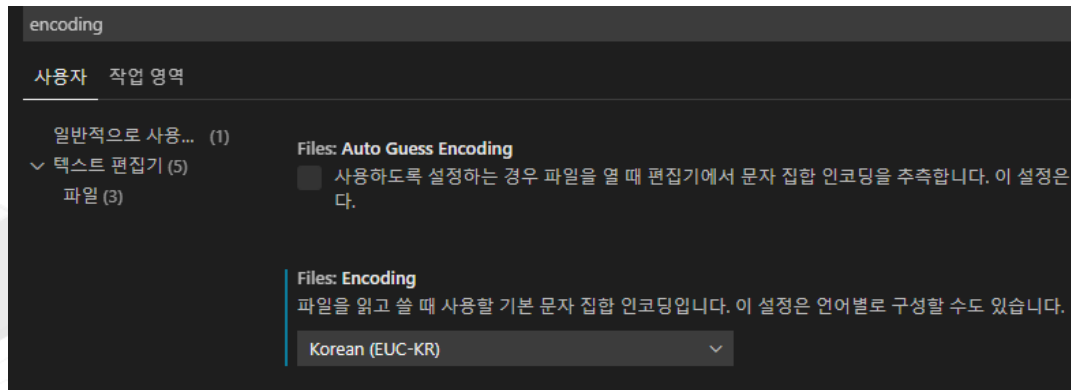
```
PS E:\WPython> py .\Wex07_1.py  
내가만든 파일쓰기
```



2 파일처리

참고: 기본캐릭터셋 utf-8 변경: Korean(EUC-KR)변경

✓ Encoding 검색





2 파일처리

with 키워드

✓ 조건문과 반복문 들어가다 보면 파일을 열고서 닫지 않는 실수 하는 경우 생길 수 있음

✓ with 구문 종료 시 **파일을 자동으로 닫음**

```
with open("test2.txt", "w") as f:  
    f.write("파일쓰기2")
```

```
with open("test2.txt", "r") as f:  
    print(f.read())
```

test2 - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
파일 쓰기2

```
PS E:\Python> py .\ex07_1.py  
파일 쓰기2
```

기상 빅데이터 정보 수집 처리

01

모듈 개념과 종류





1 표준모듈

모듈 개념

✔ 모듈 (module) : 기능 확장

» 코드를 분리하고 공유하는 기능

» 이미 만들어 둔 함수와 변수를 사용할 수 있는 기능

구분	주요 내용
표준 모듈	파이썬에 기본적으로 내장되어있는 모듈
외부 모듈 external module	- 파이썬이 기본적으로 제공하는 것이 아닌 다른 사람들이 만들어 배포하는 모듈 - 별도 다운 받아서 사용하는 모듈



1 표준모듈

표준모듈 읽기

```
op = __import__("math")  
print(op.pi)
```

3.141592653589793

✓ Import 구문

```
import math  
print(math.pi)
```

✓ Import as 구문

» 모듈의 이름이 너무 길어 짧게 줄여 사용하고 싶은 경우

```
import math as m  
print(m.pi)
```



1 표준모듈

표준모듈 읽기

✓ from 구문

- » 0다양한 함수를 계속해서 입력하는 것의 비효율성
- » ‘가져오고 싶은 변수 또는 함수’에 여러 개의 변수 또는 함수 입력 가능

✓ from import 구문

- » 이를 통해 가져온 기능은 math 붙이지 않고도 사용할 수 있음

```
from math import pi  
print(pi)
```



1 표준모듈

표준모듈 읽기

✓ from import * 구문 : 모듈을 여러 개 사용하는 경우

```
from math import *  
print(pi)
```





1 표준모듈

표준모듈 : sys

Python Runtime Services

» sys — System-specific parameters and functions

```
import sys
```

```
print(sys.copyright)
```

```
#명령 매개변수
```

```
print(sys.argv)
```

Copyright (c) 2001–2021 Python Software Foundation.
All Rights Reserved.

```
py .\wex08_1.py 1 2 3
```

```
['.\\wex08_1.py', '1', '2', '3']
```



1 표준모듈

표준모듈 : datetime

datetime 모듈

» 날짜와 시간 관련된 모듈

» 날짜 형식 만들 때 자주 사용되는 코드들 구성

```
import datetime
```

```
now = datetime.datetime.now()  
print(now)
```

예시 : 사용자마다 결과 값 다름

```
from datetime import datetime
```

2021-04-19 14:08:54.959740

```
now = datetime.now()  
print(now)
```



1 표준모듈

▶ 표준모듈 : time

✔ time 모듈

» 시간과 관련된 기능





1 표준모듈

⚙ 표준모듈 : time

✔ time.sleep() 함수

» 특정 시간 동안 코드 진행을 정지

» 정지하고 싶을 시간을 초 단위로 입력

```
import time  
print("잠시2초간 대기합니다")
```

#대기

```
time.sleep(2)  
print("정상작동합니다.")
```



1 표준모듈

표준모듈 : **urllib** 모듈

✓ URL 다루는 라이브러리

» `urlopen()` 함수 : URL 주소의 페이지 열기

```
from urllib import request
```

홈페이지 읽기

```
target = request.urlopen("http://www.naver.com")  
content = target.read()  
print(content)
```

바이너리 데이터

```
b'Wn<!doctype html>                <html lang="ko" data-dark="false"> <head> <meta charset="utf-8">  
<title>NAVER</title> <meta http-equiv="X-UA-Compatible" content="IE=edge"> <meta name="viewport"  
content="width=1190"> <meta name="apple-mobile-web-app-title" content="NAVER"/> <meta name="robots"  
content="index,nofollow"/> <meta name="description"
```



1 표준모듈

표준모듈 : os 모듈

✓ 운영체제와 관련된 기능 가진 모듈

✓ 새로운 폴더 만들거나 폴더 내부 파일 목록 보는 등

» `Os.listdir(폴더 경로)` : 폴더 내부의 모든 것을 리스트로 리턴

```
import os
```

```
print("현재 운영체제: ", os.name)
print("현재 폴더: ", os.getcwd())
print("현재폴더 내부요소: ", os.listdir())
```

현재 운영체제: nt
현재 폴더: E:\WPython
현재폴더 내부요소: ['bmi.txt', 'ex04', 'ex05', 'ex06', 'ex07_1.py', 'ex07_2.py', 'ex07_3.py', 'ex08_1.py', 'test.txt', 'test2.txt']

사용자 마다 결과 값 다름



1 표준모듈

▶ 표준모듈 : os 모듈

모듈 읽기

```
import os
```

현재 폴더 파일/폴더 출력

```
output = os.listdir(".")
print("os.listdir()", output)
print()
```

현재 폴더 파일/폴더 구분

```
for i in output:
    if os.path.isdir(i):
        print("폴더:", i)
    else:
        print("파일:", i)
```

폴더 일 경우 True

```
os.listdir() ['bmi.txt', 'ex04', 'ex05', 'ex06',
'ex07_1.py', 'ex07_2.py', 'ex07_3.py', 'ex08_1.py',
'test.txt', 'test2.txt']
```

파일: bmi.txt

폴더: ex04

폴더: ex05

폴더: ex06

파일: ex07_1.py

파일: ex07_2.py

파일: ex07_3.py

파일: ex08_1.py

파일: test.txt

파일: test2.txt



2 모듈만들기

➤ 모듈(module)

- ✓ 변수, 함수, 클래스 등을 모아 놓은 스크립트 파일
- ✓ 모듈은 간단한 기능을 담을 때 사용

➤ 패키지(package)

- ✓ 여러 모듈을 묶은 것, 패키지는 기능들이 모듈 여러 개로 나누어져 있고, 관련된 모듈끼리 모여 있는 형태
- ✓ 패키지는 코드가 많고 복잡할 때 사용



2 모듈만들기

- ▶ 파이썬 스크립트 작성 시 매번 비슷한 클래스와 함수를 작성한다면 코드도 길어지고 중복
- ✔ 공통되는 부분을 빼내서 모듈과 패키지로 생성
- ✔ 이후에는 코드를 다시 만들지 않고 모듈과 패키지만 가져와서 사용하면 편리





2 모듈만들기

square2.py

```
a = 2      # 변수  
  
def square(n):    # 함수  
    return a ** n
```

#쥬피터노트북 테스트

```
import square2  
print(square2.a)          # 모듈.변수 : 모듈 변수 사용  
print(square2.square(3))  # 모듈.함수() : 모듈 함수 사용
```

from import로 변수와 함수를 가져온 뒤 모듈 이름 없이 사용

```
from square2 import a, square  
print(a)  
print(square(3))
```



2 모듈만들기



`__name__`

- ✓ 모듈의 이름이 저장되는 변수
- ✓ `import`로 모듈을 가져왔을 때 모듈의 이름이 들어 감
- ✓ 파이썬 인터프리터로 스크립트 파일을 직접 실행했을 때는 모듈의 이름이 아니라 `'__main__'` 들어 감
- ✓ `__name__` 변수를 통해 현재 스크립트 파일이 시작점인지 모듈인지 판단



2 모듈만들기

▶ ‘__main__’

- ✓ 프로그램의 시작점 (entry point)
- ✓ 파이썬 인터프리터가 최초로 실행한 스크립트 파일의 `__name__`에 `'__main__'`이 들어 감





2 모듈만들기

➤ 모듈과 시작점

hello.py

```
print('hello 모듈 시작')  
print('hello.py __name__:', __name__) # __name__ 변수 출력  
print('hello 모듈 끝')
```

#import 모듈 가져오면 스크립트 한번 실행
import hello

print("main __name__ : ", __name__)



```
hello 모듈 시작  
hello.py __name__: hello  
hello 모듈 끝
```

```
main __name__ : __main__
```



2 모듈만들기

➤ 모듈과 시작점

✔ 파이썬 인터프리터로 실행하면 10, 20의 합 출력

» 스크립트 파일을 모듈로 사용할 때는 `calc.add`,
`calc.mul`처럼 함수만 사용하는 것이 목적이므로 10,
20의 합과 곱을 출력하는 코드는 필요 없음





2 모듈만들기

모듈과 시작점

✓ 파이썬 인터프리터로 실행하면 10, 20의 합 출력

cal.py

```
def add(a, b):  
    return a + b
```

프로그램의 시작점일 때만 아래 코드 실행

```
if __name__ == '__main__':  
    print(__name__)  
    print(add(10, 20))
```

실행결과

```
__main__  
30
```

main.py

모듈로 가져왔을 때는 출력안 됨

```
import calc  
calc.add(10,20)
```

기상 빅데이터 정보 수집 처리

02

BMI 계산





BMI 계산하기



BMI를 이용한 비만도 계산

- ✓ 자신의 몸무게를 키의 제곱으로 나누기 kg/m^2
- ✓ BMI가 18.5 이하면 저체중, 18.5 ~ 22.9 사이면 정상, 23.0 ~ 24.9 사이면 과체중, 25.0 이상 비만 판정

» 계산식 : $73 / (1.7 \times 1.7) = 25.26 \rightarrow \text{비만}$





BMI 계산하기



BMI를 이용한 비만도 계산



예 몸무게 73kg, 키 170cm

n = "홍길동"

w = 73

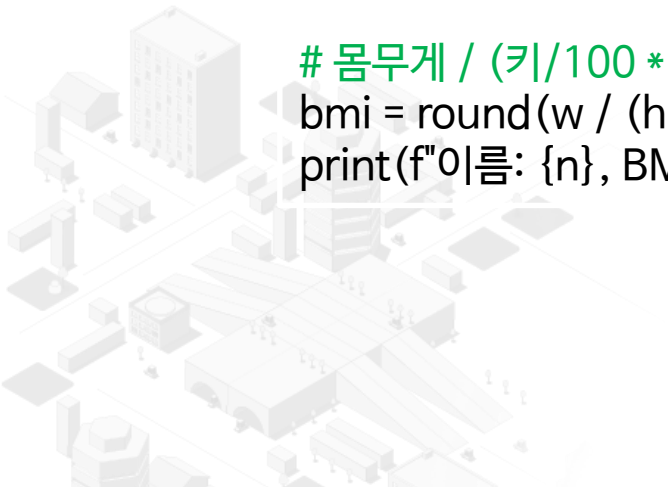
h = 170

몸무게 / (키/100 * 키/100)

bmi = round(w / (h/100 * h/100), 2)

print(f"이름: {n}, BMI : {bmi}")

이름: 홍길동, BMI : 25.26





BMI 계산하기



판정결과

n = "홍길동"

w = 73

h = 170

몸무게 / (키/100 * 키/100)

bmi = round(w / (h/100 * h/100), 2)

if bmi < 18.5:

result = "저체중"

elif bmi < 23:

result = "정상"

elif bmi < 25:

result = "과체중"

else:

result = "비만"

print(f"이름: {n}, BMI : {bmi}, 판정결과: {result}")

BMI 판정기준

18.5 이하 : 저체중

18.5 ~ 22.9 : 정상

23.0 ~ 24.9 : 과체중

25.0 이상 : 비만

이름: 홍길동, BMI : 25.26, 판정결과: 비만



BMI 계산하기



사용자 입력

```
n = input(">>> 이름 입력: ")  
w = input(">>> 몸무게 입력: ")  
h = input(">>> 키 입력: ")
```

```
# 몸무게 / (키/100 * 키/100)
```

```
bmi = round(float(w) / (float(h)/100 * float(h)/100), 2)
```

```
if bmi < 18.5:  
    result = "저체중"  
elif bmi < 23:  
    result = "정상"  
elif bmi < 25:  
    result = "과체중"  
else:  
    result = "비만"
```

```
print(f"이름: {n}, BMI : {bmi}, 판정결과: {result}")
```



BMI 계산하기



함수 생성과 파일 출력하기

```
def BmiTest(bmi):  
    if bmi < 18.5:  
        result = "저체중"  
    elif bmi < 23:  
        result = "정상"  
    elif bmi < 25:  
        result = "과체중"  
    else:  
        result = "비만"  
    return result
```

```
n = input(">>> 이름 입력: ")  
w = input(">>> 몸무게 입력: ")  
h = input(">>> 키 입력: ")
```

```
# 몸무게 / (키/100 * 키/100)
```

```
bmi = round(float(w) / (float(h)/100 *  
float(h)/100), 2)
```

```
result = BmiTest(bmi)
```

```
print(f"이름: {n}, BMI : {bmi}, 판정결과: {result}")
```

```
with open("bmi.txt", "w") as f:
```

```
    f.write(f"이름: {n}, BMI : {bmi}, 판정결과:  
{result}")
```



BMI 계산하기



단계 1: 100명 사람 이름, 몸무게, 키 데이터 임의 생성

단계 1 : 임의 데이터 생성하기

```
import random
```

```
t = list("가나다라마바사아자차카타파하")
```

```
with open("bmi.txt", "w") as f:
```

```
    for i in range(100):
```

```
        n = random.choice(t) + random.choice(t) + random.choice(t)
```

```
        w = random.randint(40,100)
```

```
        h = random.randint(130,200)
```

```
        f.write(f"{n}, {w}, {h} \n")
```



bmi - Windows 메모장

파일(F) 편집(E) 서식(O) 보:

아카차, 68, 177

차차바, 49, 176

차하나, 77, 162

하마카, 70, 189

카타바, 96, 189

나가파, 41, 164

파다마, 45, 188

마다사, 51, 178

타다자, 85, 178

라다아, 49, 182

나가파, 55, 167

타차아, 94, 167

카나카, 99, 176



BMI 계산하기



단계 2 : 키와 몸무게로 비만도 계산



이름, BMI출력하기

```
with open("bmi.txt", "r") as f:
    for i in f:
        (n, w, h) = i.strip().split(",")
        bmi = float(w) / (float(h)/100 * float(h)/100)
        print(f"이름: {n}, bmi : {bmi}")
```



strip() 함수: 문자열 양 옆의 공백 제거



split() 함수: 문자열을 특정한 문자로 자르고
결과는 리스트로 출력



BMI 계산하기



단계 3 : 비만도 판정

```
def BmiTest(bmi):  
    if bmi < 18.5:  
        result = "저체중"  
    elif bmi < 23:  
        result = "정상"  
    elif bmi < 25:  
        result = "과체중"  
    else:  
        result = "비만"  
    return result
```

판정표

<18.5 저체중

<23 정상

<25 과체중

>25 비만

이름: 사카하, BMI: 24.67, 결과: 정상체중

이름: 다가하, BMI: 25.88, 결과: 과체중

이름: 바자자, BMI: 25.05, 결과: 과체중

```
with open("bmi.txt", "r") as f:  
    for i in f:  
        (n, w, h) = i.strip().split(",")  
        bmi = round(float(w) / (float(h)/100 * float(h)/100), 2)  
        print(f"이름: {n}, bmi : {bmi}, 판정결과: {BmiTest(bmi)}")
```

기상 빅데이터 정보 수집 처리

03

기상청 날씨예보 정보 크롤링



1 외부모듈 : BeautifulSoup4

➤ pip (Python Installs Packages)

✔ 특정 버전 모듈 설치 및 제거 등





1 외부모듈 : BeautifulSoup4

⚙ BeautifulSoup4 모듈: 웹 사이트 태그 해석

- ✓ 수집 하고자 하는 페이지의 URL을 확인
- ✓ 파이썬의 작은 브라우저 requests 를 통해 URL에 접근
- ✓ response.`status_code` 가 200 OK 라면 정상 응답
- ✓ request의 response 값에서 response. text 만 받아옴
- ✓ html 텍스트를 받아왔다면 bs(response.text, 'html.parser') 로 html 해석



1 외부모듈 : BeautifulSoup4

⚙ BeautifulSoup4 모듈: 웹 사이트 태그 해석

- ✓ soup.select 를 통해 원하는 태그에 접근
- ✓ 목록을 먼저 받아옴
- ✓ 목록에서 행을 하나씩 가져옴
- ✓ 행을 모아서 데이터프레임으로 만듦





2 Beautiful Soup 모듈

➤ HTML과 XML 파일로부터 데이터를 뽑아내기 위한 파이썬 라이브러리

➤ <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

Beautiful Soup 4.9.0 documentation » Beautiful Soup Documentation

Table of Contents

- Beautiful Soup Documentation
 - Getting help
- Quick Start
- Installing Beautiful Soup
 - Problems after installation
 - Installing a parser
- Making the soup
- Kinds of objects
 - Tag
 - Name
 - Attributes
 - Multi-valued attributes
 - NavigableString
 - BeautifulSoup
 - Comments and other special strings

Beautiful Soup Documentation

Beautiful Soup is a Python library for pulling data out of HTML and XML files. It works with your favorite parser to provide idiomatic ways of navigating, searching, and modifying the parse tree. It commonly saves programmers hours or days of work.

These instructions illustrate all major features of Beautiful Soup 4, with examples. I show you what the library is good for, how it works, how to use it, how to make it do what you want, and what to do when it violates your expectations.

This document covers Beautiful Soup version 4.9.3. The examples in this documentation should work the same way in Python 2.7 and Python 3.8.

You might be looking for the documentation for Beautiful Soup 3. If so, you should know that Beautiful Soup 3 is no longer being developed and that support for it will be dropped on or after December 31, 2020. If you want to learn about the differences between Beautiful Soup 3 and Beautiful Soup 4, see [Porting code to BS4](#).





2 Beautiful Soup 모듈

⚙️ Beatifulsoup 외부모듈 설치

✔️ easy_install BeautifulSoup4

pip install beatifulsoup4

pip 설치시 에러 발생

```
PS E:\Python> pip install beatifulsoup4
ERROR: Could not find a version that satisfies the requirement beatifulsoup4
ERROR: No matching distribution found for beatifulsoup4
```

```
PS E:\Python> easy_install BeautifulSoup4
```

Easy_install 이용

pip uninstall BeautifulSoup4

easy_install BeautifulSoup4



2 Beautiful Soup 모듈

Beatifulsoup 외부모듈 설치

✔ 오류: pip version 업그레이드 하기

```
문제  출력  디버그 콘솔  터미널
2: Python Debug Consc v + [] 🗑 ^ x

PS E:\Python> pip install beatifulsoup4
ERROR: Could not find a version that satisfies the requirement beatifulsoup4 (from versions: none)
ERROR: No matching distribution found for beatifulsoup4
WARNING: You are using pip version 20.2.3; however, version 21.0.1 is available.
You should consider upgrading via the 'c:\users\pc\appdata\local\programs\python\python39\python.exe -m pip install --upgrade pip' command.
PS E:\Python> 
```

python -m pip install --upgrade pip

```
PS E:\Python> python -m pip install --upgrade pip
Collecting pip
  Downloading pip-21.0.1-py3-none-any.whl (1.5 MB)
    | 1.5 MB 1.6 MB/s
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 20.2.3
    Uninstalling pip-20.2.3:
      Successfully uninstalled pip-20.2.3
  Successfully installed pip-21.0.1
```



2 Beautiful Soup 모듈

Beatifulsoup 외부모듈 설치

✓ whl 파일 다운로드

» <https://pypi.org/project/beautifulsoup4/#files>

Download files

Download the file for your platform. If you're not sure which to choose, learn more about [installing packages](#).

Filename, size	File type	Python version	Upload date	Hashes
beautifulsoup4-4.9.3-py2-none-any.whl (115.9 kB)	Wheel	py2	Oct 4, 2020	View
beautifulsoup4-4.9.3-py3-none-any.whl (115.8 kB)	Wheel	py3	Oct 4, 2020	View
beautifulsoup4-4.9.3.tar.gz (376.0 kB)	Source	None	Oct 4, 2020	View



2 Beautiful Soup 모듈

실습 폴더에 복사

터미널에 아래 명령어 실행

```
PS D:\Labs> python -m pip install beautifulsoup4-4.9.3-py3-none-any.whl
Processing d:\labs\beautifulsoup4-4.9.3-py3-none-any.whl
Collecting soupsieve>1.2
  Downloading soupsieve-2.2.1-py3-none-any.whl (33 kB)
Installing collected packages: soupsieve, beautifulsoup4
Successfully installed beautifulsoup4-4.9.3 soupsieve-2.2.1
```

python -m pip install beautifulsoup4-4.9.3-py3-none-any.whl

파일 이름 확인



2 Beautiful Soup 모듈

Beautiful Soup 모듈 테스트

```
html_doc = """
<html>
  <head>
    <title> TEST PAGE </title>
  </head>
  <body>
    <h1>빅데이터프로그래밍</h1>
  </body>
</html>
```

Html문서시작

```
"""
Html문서종료
```

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

print(soup.select("h1"))
print(soup.select("h1")[0].string)
```

[<h1>빅데이터프로그래밍</h1>]
빅데이터프로그래밍



3 기상청 RSS

▶ RSS (Really Simple Syndication, Rich Site Summary)

- ✔ 콘텐츠 업데이트가 자주 일어나는 웹사이트에서, 쉽게 구독자들에게 제공하기 위해 XML을 기초로 만들어진 데이터 형식
- ✔ RSS서비스를 이용하면 업데이트된 정보를 찾기 위해 홈페이지에 일일이 방문하지 않아도 업데이트 될 때마다 빠르고 편리하게 확인할 수 있음





3 기상청 RSS

기상청 날씨누리 - 생활과산업 - 서비스

예시: 인천시 미추홀구

웹 RSS 날씨위젯

RSS란?

RSS(Really Simple Syndication, Rich Site Summary)란 블로그처럼 콘텐츠 업데이트가 자주 일어나는 웹사이트에서, 업
를 쉽게 구독자들에게 제공하기 위해 XML을 기초로 만들어진 데이터 형식입니다. RSS서비스를 이용하면 업데이트된 정보
홈페이지에 일일이 방문하지 않아도 업데이트 될 때마다 빠르게 편리하게 확인할 수 있습니다.

RSS 서비스 이용하기



동네예보 > 시간별예보

동네예보	인천광역시	검색	미추홀구	검색	옹현1.4동	검색	RSS
------	-------	----	------	----	--------	----	-----

www.weather.go.kr 내용:

Ctrl + c 를 눌러 클립보드로 복사하세요

<http://www.kma.go.kr/wid/queryDFSRSS.jsp?zone=2817754000>

확인

취소



3 기상청 RSS

urlopen() 함수로 기상청의 전국 날씨 오픈

```
from urllib import request
from bs4 import BeautifulSoup
```

```
content = request.urlopen("http://www.kma.go.kr/wid/queryDFSRSS.jsp?zone=2817754000")
soup = BeautifulSoup(content, "html.parser")
```

```
for i in soup.select("data"):
    print(i)
    print("-" * 20)
```

```
PS E:\Python> python ex08_4.py
<data seq="0">
<hour>21</hour>
<day>0</day>
<temp>11.0</temp>
<tmx>-999.0</tmx>
<tmn>-999.0</tmn>
<sky>1</sky>
<pty>0</pty>
<wfkor>맑음</wfkor>
<wfen>Clear</wfen>
<pop>0</pop>
<r12>0.0</r12>
<s12>0.0</s12>
<ws>2.6</ws>
<wd>5</wd>
<wdkor>남서</wdkor>
<wden>SW</wden>
<reh>75</reh>
<r06>0.0</r06>
<s06>0.0</s06>
</data>
-----
```

```
▼ <data seq="0">
  <hour>21</hour>
  <day>0</day>
  <temp>11.0</temp>
  <tmx>-999.0</tmx>
  <tmn>-999.0</tmn>
  <sky>1</sky>
  <pty>0</pty>
  <wfkor>맑음</wfkor>
  <wfen>Clear</wfen>
  <pop>0</pop>
  <r12>0.0</r12>
  <s12>0.0</s12>
  <ws>2.6</ws>
  <wd>5</wd>
  <wdkor>남서</wdkor>
  <wden>SW</wden>
  <reh>75</reh>
  <r06>0.0</r06>
  <s06>0.0</s06>
</data>
```



3 기상청 RSS

urlopen() 함수로 기상청의 전국 날씨 오픈

```
from urllib import request
from bs4 import BeautifulSoup

content =
request.urlopen("http://www.kma.go.kr/wid/queryDFSRSS.jsp?zone=2817754000")
soup = BeautifulSoup(content, "html.parser")

print("장소: ", soup.select("category") [0].string)
for i in soup.select("data"):
    # print(i)
    hour = i.select_one("hour").string
    temp = i.select_one("temp").string
    wfKor = i.select_one("wfKor").string
    reh = i.select_one("reh").string
    pop = i.select_one("pop").string
    print(f"시간: {hour}")
    print(f"시간: {} - {}".format(int(hour)-3, hour))
    print(f"온도: {temp}")
    print(f"날씨: {wfKor}")
    print(f"강수확률: {pop} %")
    print(f"습도: {reh} %")
    print("-"* 20)
```

장소: 인천광역시 미추홀구 용현1,4동

시간: 18

시간: 15 - 18

온도: 25.0

날씨: 구름 많음

강수확률: 10%

습도: 60%

시간: 21

시간: 18 - 21

온도: 22.0

날씨: 구름 많음

강수확률: 10%

습도: 75%
